

Design and Instructions of the Contact Management System

Justus Redlin, Linus Jahn

February 9, 2026

Contents

1	Design Decisions	3
1.1	Project level design	3
1.2	CLI design	3
1.3	ContactManager design	5
1.4	Contacts design	5
1.5	Logger design	5
1.6	Regular Expression	5
2	How to Run the Program	6
2.1	Option A — Run from the ZIP Submission	6
2.2	Option B — Run from GitHub	6
2.3	Running the Unit Tests	7
3	Tests we executed	8
3.1	Testing the Package	8
3.2	Testing the CLI	10

List of Figures

1	Complete class diagramm	3
2	Program Flow with Focus on File IO and Concurrency	4
3	Main menu	10
4	Choosing the contact type	11
5	Prompting the required data	11
6	Listing contacts by their group	11
7	Saving contacts upon exit	11
8	A new session loads the saved contacts	12

1 Design Decisions

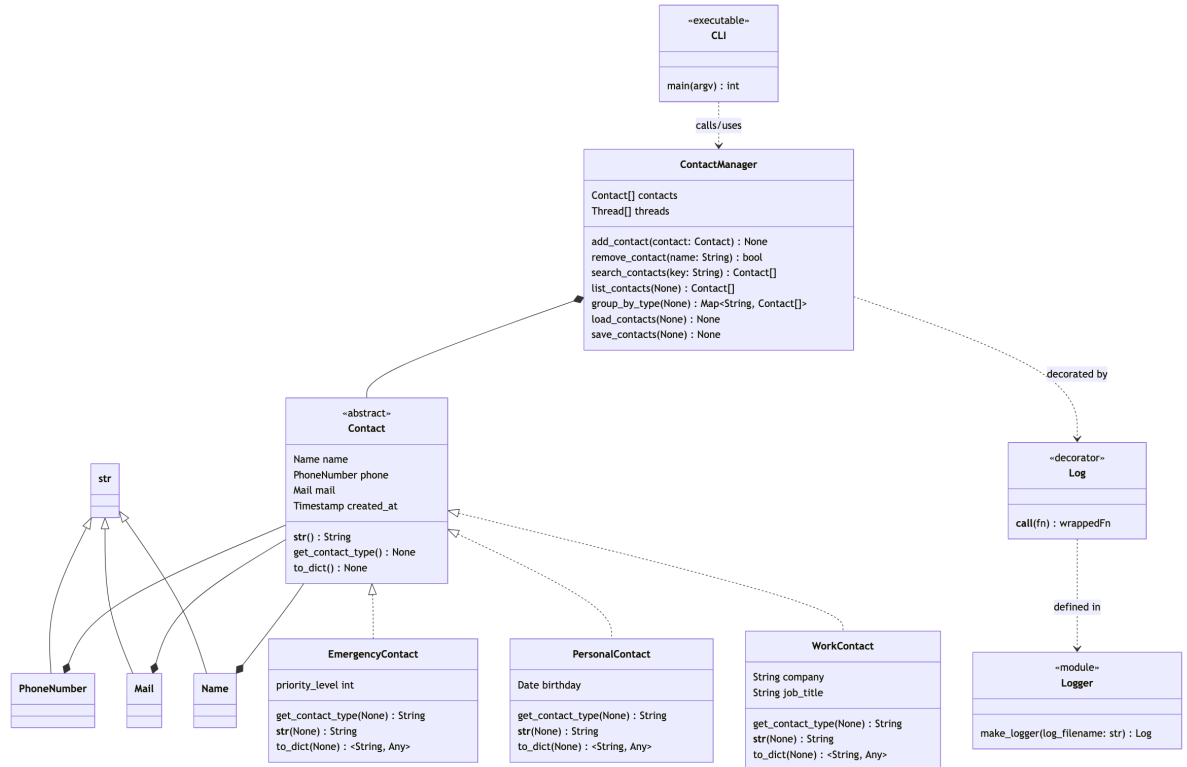


Figure 1: Complete class diagram

1.1 Project level design

- Installable package due to `pyproject.toml`
- Installable command line `executable` script
- Persistent state (json) stored in `user data` directory
- Unit tests via `pytest`

1.2 CLI design

- Dialog based on `input()`
- Loop based

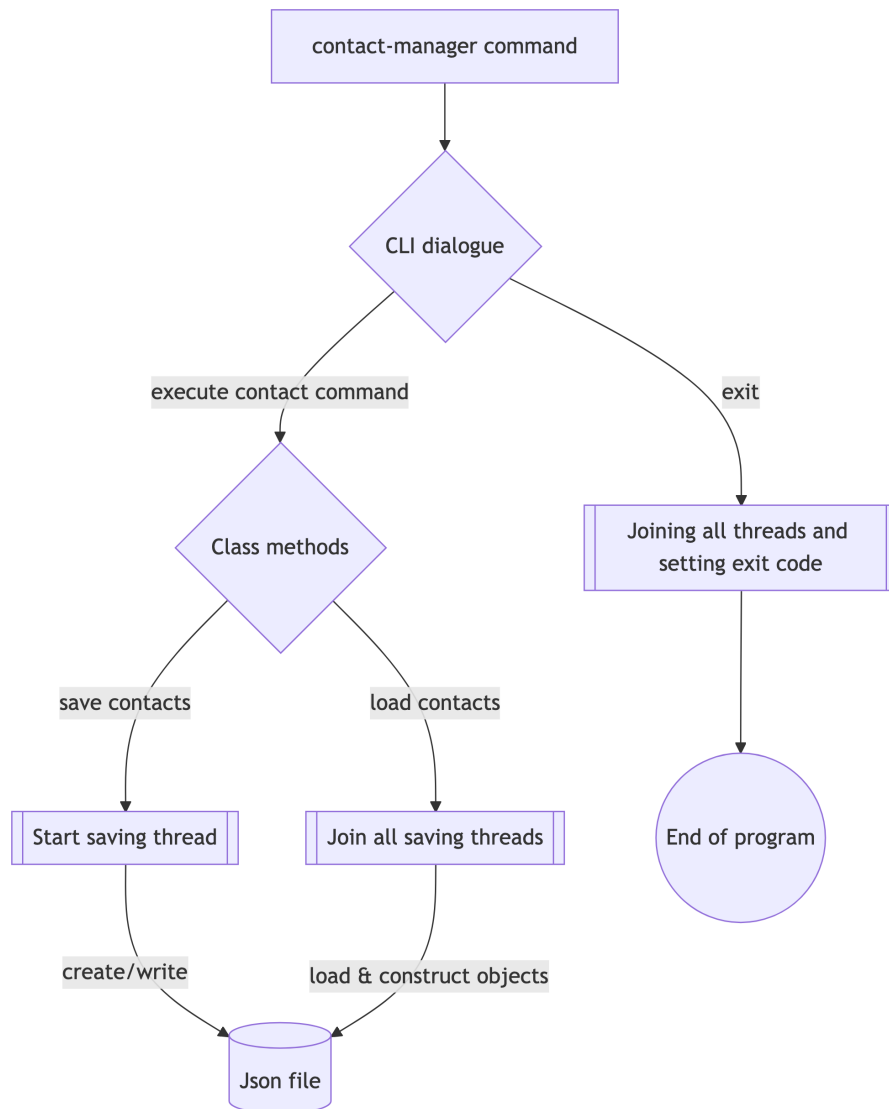


Figure 2: Program Flow with Focus on File IO and Concurrency

- Used (higher order) functions to unify Exception handling and input checking

1.3 ContactManager design

- Robustness: Create file if not exists
- Robustness: Reset file if corrupted
- Concurrency: Define `contacts_state_lock` to save contact state of call time `_contacts`.
- Concurrency: Defined `file_write_lock` to avoid race condition between threads
- Concurrency: Threads join only when needed

1.4 Contacts design

- Abstraction: Deserialization (from string to objects) of attributes performed in the constructor
- `to_dict` returns the object needed to reconstruct the instance

1.5 Logger design

- Capsulation: subpackage
- Architecture: Decorator factory takes a callable log function as an argument
- Architecture: log function factory configures file path via its closure

1.6 Regular Expression

- Architecture: Regex checks in constructor of `PhoneNumber` `Mail` `Name` classes
- Architecture: Regex classes inherit from string, to treat objects as an instance thereof
- Inheritance done via `__new__`
- Architecture: Regex classes used in the constructor of `Contact`

2 How to Run the Program

The Contact Management CLI can be executed either from the provided ZIP submission or by cloning the GitHub repository. The steps below describe the minimal setup required to run the program on Windows.

2.1 Option A — Run from the ZIP Submission

1. Unzip the project to a folder of your choice.
2. Open a terminal in the project root directory (the folder containing `pyproject.toml`).
3. (Recommended) Create and activate a virtual environment:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
```

4. Install the package (editable install):

```
py -m pip install -e .
```

5. Start the CLI:

```
contact-manager
```

2.2 Option B — Run from GitHub

1. Clone the repository and switch into the project directory:

```
git clone https://github.com/xXuserseXx/Provadis-Personal-Contact-Management-System
cd Provadis-Personal-Contact-Management-System
```

2. Create and activate a virtual environment, install the package, and run the CLI:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
py -m pip install -e .
contact-manager
```

Expected result: The command `contact-manager` starts an interactive, menu-driven CLI. Contacts are loaded automatically on startup; saving writes the contact data to the OS-specific user data directory.

Note (PowerShell execution policy): If activating the virtual environment is blocked, run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

and then retry:

```
.\.venv\Scripts\Activate.ps1
```

2.3 Running the Unit Tests

The unit tests are located in the `./tests` folder and can be executed with `pytest`. After setting up and activating the virtual environment as described above, install the development dependencies and run the test suite:

```
py -m pip install -e "[dev]"  
py -m pytest
```

3 Tests we executed

3.1 Testing the Package

The project contains a small but focused unit test suite in `./tests`. With these tests we checked that the most important parts of the package work as intended:

- contacts can be created and converted into a dictionary/JSON-friendly format (so they can be saved),
- invalid inputs (e.g., malformed email addresses or phone numbers) are rejected early with clear errors,
- the `ContactManager` can add, list, search, group, remove contacts, and can save/load them again without losing information.

Selected unit tests (significant samples).

```
def test_regular_expression_value_objects_validate_inputs():
    """Mail, PhoneNumber, and Name should reject invalid inputs with
    ↪ clear errors."""
    # Valid cases are accepted.
    assert Mail("luis.gaertner@stud-provadis.de") ==
    ↪ "luis.gaertner@stud-provadis.de"
    assert PhoneNumber("+49 151 234 567") == "+49 151 234 567"
    assert Name("Justus Max-Redlin") == "Justus Max-Redlin"

    # Invalid formats should raise a ValueError.
    with pytest.raises(ValueError):
        Mail("bad@address@invalid")
    with pytest.raises(ValueError):
        PhoneNumber("not-a-number")
    with pytest.raises(ValueError):
        Name("Justus Redlin")

    # Invalid types should raise a TypeError.
    with pytest.raises(TypeError):
        Mail(123)
    with pytest.raises(TypeError):
        PhoneNumber(123)
    with pytest.raises(TypeError):
        Name(123)
```

What this test shows: Valid values are accepted, but wrong values are rejected with the right exception type: `TypeError` for wrong data types and `ValueError` for wrong formats. This makes it harder for invalid data to enter the system.

```
def test_contact_manager_add_search_group_remove():
    """ContactManager should manage the contact list as expected."""
    manager = ContactManager()
```



```

personal = PersonalContact("Justus", "+49 151 234 567",
    ↳ "gg.jr@example.de", date(1999, 12, 30))
work = WorkContact("Nadine", "0049-151-123-456",
    ↳ "nadine@example.com", "Acme", "Engineer")

# Adding contacts should make them visible in the list.
manager.add_contact(personal)
manager.add_contact(work)
assert manager.list_contacts() == [personal, work]

# Search should find contacts by any searchable field.
search_results = manager.search_contacts("acme")
assert search_results == [work]

# Grouping should bucket by contact type.
grouped = manager.group_by_type()
assert grouped["PersonalContact"] == [personal]
assert grouped["WorkContact"] == [work]

# Removing by name should return a boolean and update the list.
assert manager.remove_contact("Justus") is True
assert manager.list_contacts() == [work]
assert manager.remove_contact("Missing") is False

```

What this test shows: It checks the basic contact management features in one realistic scenario.

```

def
↳ test_contact_manager_saves_and_loads_round_trip(temp_contact_storage):
    """Saving and loading should preserve the contact data and types."""
    manager = ContactManager()
    personal = PersonalContact("Justus", "+49 151 234 567",
        ↳ "gg.jr@example.de", date(1999, 12, 30))
    emergency = EmergencyContact("Nora", "+1 415 555 0101",
        ↳ "nora@example.com", 3)
    manager.add_contact(personal)
    manager.add_contact(emergency)

    # Save contacts to the temporary file.
    manager.save_contacts()

    # Load the contacts into a new manager and confirm types/fields.
    reloaded_manager = ContactManager()
    # As the threads are shared among instances and load_contracts joins
    ↳ all threads by itself
    # There is no need to call join additionally.
    reloaded_manager.load_contacts()
    reloaded_contacts = reloaded_manager.list_contacts()

    assert len(reloaded_contacts) == 2
    assert any(isinstance(contact, PersonalContact) for contact in
        ↳ reloaded_contacts)

```

```

assert any(isinstance(contact, EmergencyContact) for contact in
↪   reloaded_contacts)

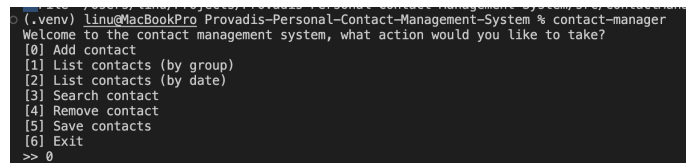
# Verify that the data survived the round-trip.
names = {contact.name for contact in reloaded_contacts}
assert names == {"Justus", "Nora"}

```

What this test shows: It checks that saving and loading really works end-to-end

3.2 Testing the CLI

The CLI (`contact-manager`) was tested **manually** to check the actual user flow (menus, prompts, and the expected actions such as adding contacts, listing/searching contacts, and saving). In this subsection, we provide **screenshots** of an **exemplary user interaction**, with short captions describing what happens.



```

(.venv) linu@MacBookPro Provadis-Personal-Contact-Management-System % contact-manager
Welcome to the contact management system, what action would you like to take?
[0] Add contact
[1] List contacts (by group)
[2] List contacts (by date)
[3] Search contact
[4] Remove contact
[5] Save contacts
[6] Exit
>> 0

```

Figure 3: Main menu

```

What kind of contact would you like to add?
[0] personal contact
[1] emergency contact
[2] work contact
>> 0

```

Figure 4: Choosing the contact type

```

What is the name of the contact?
>> Justus Max Redlin
What is the phone number of the contact?
>> 0151 55955
What is the email of the contact?
>> jj.gg@gmail.com
What is the birthday of the contact?
Provide the date in the following format: DD.MM.YYYY .
>> 01.01.1999

```

Figure 5: Prompting the required data

```

Welcome to the contact management system, what action would you like to take?
[0] Add contact
[1] List contacts (by group)
[2] List contacts (by date)
[3] Search contact
[4] Remove contact
[5] Save contacts
[6] Exit
>> 1
Your personal contacts:
Name: Justus Max Redlin, Phone: 0151 55955, Email: jj.gg@gmail.com, Birthday: 1999-01-01

Your emergency contacts:
Emergency Contact, Name: John, Phone: +34 151 4944, Email: john.schmoe@gmail.com, priority level: 1
Emergency Contact, Name: Linus, Phone: +49 12549444, Email: lin.jj@mail.com, priority level: 5

Your work contacts:
Name: John, Phone: 145 9554, Email: peckert.john@mail.com, Company: peckertInc, Title: accountant

```

Figure 6: Listing contacts by their group

```

Welcome to the contact management system, what action would you like to take?
[0] Add contact
[1] List contacts (by group)
[2] List contacts (by date)
[3] Search contact
[4] Remove contact
[5] Save contacts
[6] Exit
>> 6
Saving your current contacts...
Bye!

```

Figure 7: Saving contacts upon exit

```
(.venv) Linu@MacBookPro Provadis-Personal-Contact-Management-System % contact-manager
Welcome to the contact management system, what action would you like to take?
[0] Add contact
[1] List contacts (by group)
[2] List contacts (by date)
[3] Search contact
[4] Remove contact
[5] Save contacts
[6] Exit
>> 2
Created at 09.02.2026 09:19:17:
  Name: Justus Max Redlin, Phone: 0151 55955, Email: jj.gg@gmail.com, Birthday: 1999-01-01
Created at 09.02.2026 09:19:17:
  Emergency Contact, Name: John, Phone: +34 151 4944, Email: john.schmoe@gmail.com, priority level: 5
Created at 09.02.2026 09:19:17:
  Name: John, Phone: 145 9554, Email: peckert.john@mail.com, Company: peckertInc, Title: accountant
Created at 09.02.2026 09:19:17:
  Emergency Contact, Name: Linus, Phone: +49 12549444, Email: lin.jj@mail.com, priority level: 5
```

Figure 8: A new session loads the saved contacts